

Website Analytics Engineering Guide

Website Analytics GA4 Dashboard • Last changed July 23, 2026

This guide uses sanitized examples. Replace the sample site names, property IDs, OAuth client ID, and OAuth secret with your own values.

Table of Contents

- Table of Contents.....1
- Architecture Overview.....2
- Current Dashboard Surface3
- GA4 Reports Used.....3
- Authentication and Token Storage.....3
- Portable SQLite Settings4
- Designer/Editability Rules.....4
- Build and Validation.....5
- Public Repository Hygiene5

Architecture Overview

The application is a Delphi FMX desktop dashboard. Forms remain editable in the IDE. Runtime code retrieves GA4 data, maps responses into memory models, and updates existing designer controls.

Area	Primary unit	Purpose
Dashboard form	WebsiteAnalytics.MainForm.pas/.fmx	Hero, summary cards, chart paint boxes, grids, realtime panel, update timers.
GA4 access	WebsiteAnalytics.GA4DataModule.pas	Builds GA4 Data API and Realtime API requests and parses responses.
Authentication	WebsiteAnalytics.AuthenticationDataModule.pas	Desktop OAuth, loopback redirect, refresh-token persistence.
Settings	WebsiteAnalytics.SettingsDataModule.pas	Portable SQLite settings database and schema.
Models	WebsiteAnalytics.Models.pas	Snapshots, KPI summaries, trend points, location/content rows.

Current Dashboard Surface



Current surface area with sanitized data.

GA4 Reports Used

- Overview/KPI requests include users, sessions, screen/page views, engagement rate, event count, events per session, and scrolled users.
- Trend requests normalize missing dates so the chart can show a complete selected range.
- Geography requests group country, region, city, users, sessions, and engagement information.
- Content requests group page title, path, action/event, users, and event/download counts.
- Realtime requests read active users, realtime views, and the current/last activity location when GA4 returns those dimensions.

Authentication and Token Storage

The app uses a Google desktop OAuth client. On first sign-in the loopback listener receives an authorization code, exchanges it for tokens, and stores the refresh token encrypted for the current Windows user. Startup attempts a silent token refresh before showing any browser login screens.

Settings and Google Sign-in

Use your own Google desktop OAuth client. Secrets are hidden by default.

Default website	Example Site
Default date range	Last 7 days
OAuth client ID	123456789012-abc...apps.googleusercontent.com
OAuth client secret	
Saved refresh token	Encrypted with Windows DPAPI
Redirect URI	http://127.0.0.1:53682/oauth2redirect

OAuth settings are shown with dummy values and a hidden client secret.

Portable SQLite Settings

The SQLite database stores application settings, property definitions, saved grid widths, OAuth setup values, and the encrypted refresh token. Report data remains memory-only.

Privacy and Storage

Stored locally	Settings, property definitions, saved grid widths
Stored encrypted	Google refresh token via Windows DPAPI
Not stored	Fetches GA4 report rows and analytics results
Ignored by Git	databases/*.sqlite3
Tracked by Git	databases/WebsiteAnalytics.schema.sql

Storage responsibilities and Git tracking policy.

Designer/Editability Rules

- FMX forms should remain editable in RAD Studio.
- Do not add helper code or runtime UI layout code without developer approval.
- Prefer Object Inspector settings for control placement and sizing whenever practical.

- When runtime drawing is necessary for charts or tooltips, keep it limited to painting/data display, not form construction.

Build and Validation

1. Create a pre-change backup on D: before modifying project files.
2. Build Win64 and Win32 Debug with RAD Studio 37.0 MSBuild.
3. Smoke-test startup to catch FMX read errors.
4. Review Git status and stage only intended files.
5. Commit and push both the private and public repositories when the change is complete.

Public Repository Hygiene

- Never commit a real SQLite database, OAuth client secret, refresh token, or personal property IDs.
- Keep databases/WebsiteAnalytics.schema.sql tracked.
- Use dummy property IDs and redacted OAuth examples in documentation.
- Keep generated binaries and local build outputs ignored unless an explicit release artifact is being created.

Managing Properties

Use generic names in examples; enter your own GA4 numeric property IDs.

Example Site A

123456789

Example Site B

234567890

Example Site C

345678901

Sanitized property examples suitable for public documentation.